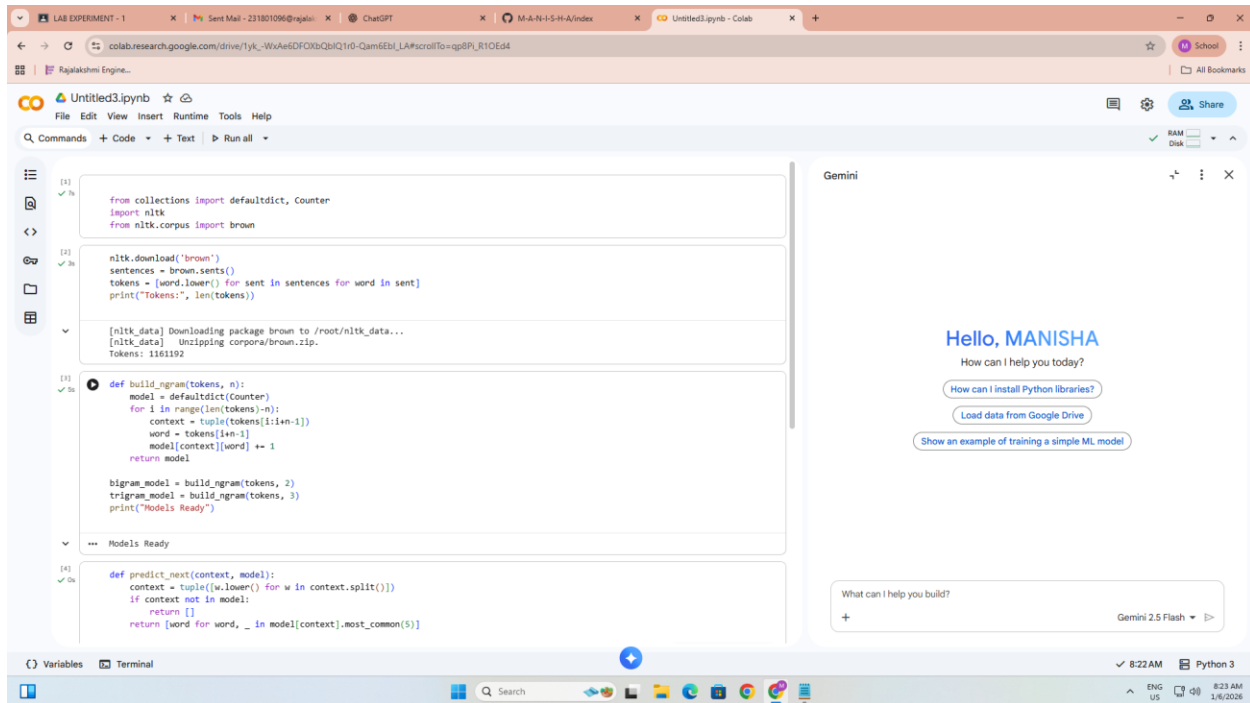


TEXT GENERATION USING N-GRAM LANGUAGE MODEL



The screenshot shows a Jupyter Notebook titled 'Untitled3.ipynb' in a web browser. The code in the notebook is as follows:

```
[1]: from collections import defaultdict, Counter
import nltk
from nltk.corpus import brown

[2]: nltk.download('brown')
sentences = brown.sents()
tokens = [word.lower() for sent in sentences for word in sent]
print("Tokens:", len(tokens))

[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data] Unzipping corpora/brown.zip.
Tokens: 1161192

[3]: def build_ngram(tokens, n):
    model = defaultdict(Counter)
    for i in range(len(tokens)-n):
        context = tuple(tokens[i:i+n-1])
        word = tokens[i+n-1]
        model[context][word] += 1
    return model

bigram_model = build_ngram(tokens, 2)
trigram_model = build_ngram(tokens, 3)
print("Models Ready")

--- Models Ready

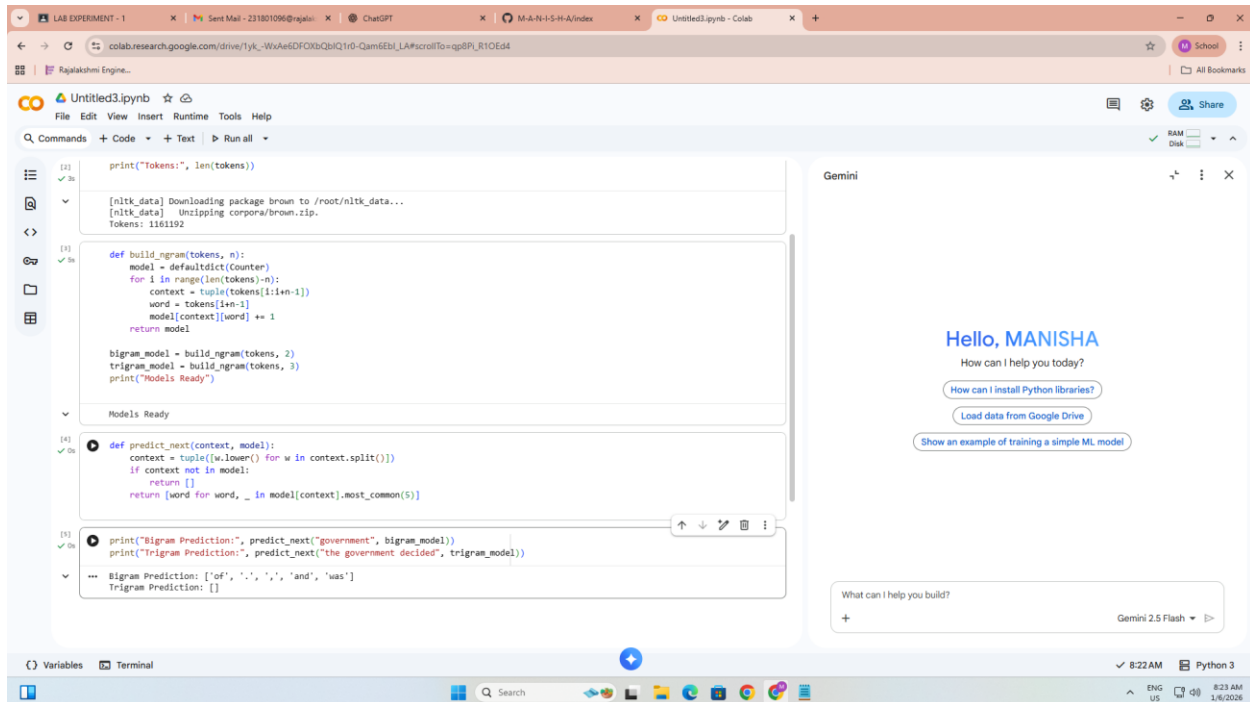
[4]: def predict_next(context, model):
    context = tuple([w.lower() for w in context.split()])
    if context not in model:
        return []
    return [word for word, _ in model[context].most_common(5)]
```

The output of the notebook shows the following:

```
Tokens: 1161192

Models Ready
```

The right sidebar shows the Gemini chat interface with the message "Hello, MANISHA" and a prompt "What can I help you build?".



The screenshot shows the same Jupyter Notebook interface, but with additional code and output. The code in the notebook is as follows:

```
[5]: print("Tokens:", len(tokens))

[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data] Unzipping corpora/brown.zip.
Tokens: 1161192

[6]: def build_ngram(tokens, n):
    model = defaultdict(Counter)
    for i in range(len(tokens)-n):
        context = tuple(tokens[i:i+n-1])
        word = tokens[i+n-1]
        model[context][word] += 1
    return model

bigram_model = build_ngram(tokens, 2)
trigram_model = build_ngram(tokens, 3)
print("Models Ready")

--- Models Ready

[7]: def predict_next(context, model):
    context = tuple([w.lower() for w in context.split()])
    if context not in model:
        return []
    return [word for word, _ in model[context].most_common(5)]

[8]: print("Bigram Prediction:", predict_next("government", bigram_model))
print("Trigram Prediction:", predict_next("the government decided", trigram_model))

--- Bigram Prediction: ['of', '.', ',', 'and', 'was']
Trigram Prediction: []
```

The output of the notebook shows the following:

```
Tokens: 1161192

Models Ready

Bigram Prediction: ['of', '.', ',', 'and', 'was']
Trigram Prediction: []
```

The right sidebar shows the Gemini chat interface with the message "Hello, MANISHA" and a prompt "What can I help you build?".